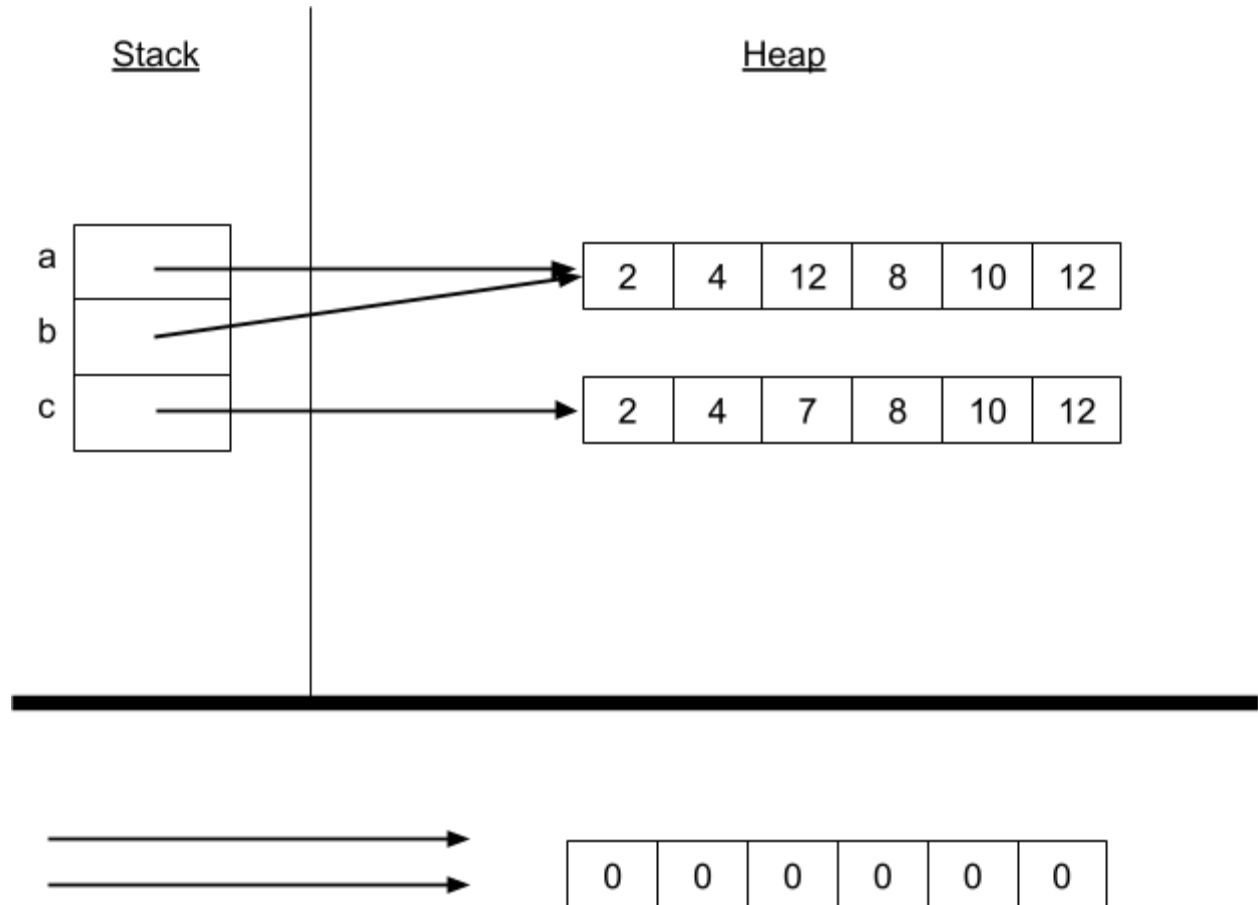


Problem Set 1, Part I

Problem 1: Memory management and arrays

1-1)



1-2)

12 12 7

Problem 2: Array practice

2-1)

```
public static boolean isSorted(int[] arr) {
    if (arr == null) {
        throw new IllegalArgumentException(
            "null array"
        );
    } else if (arr.length == 0) {
        return true;
    } else {
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i+1]) {
                return false;
            }
        }
    }

    return true;
}
```

2-2)

```
public static void scale(int[] arr, int factor) {
    if (arr == null) {
        throw new IllegalArgumentException(
            "Null array"
        );
    } else if (arr.length == 0) {
        return;
    } else {
        for (int i = 0; i < arr.length; i++) {
            arr[i] = factor * arr[i];
        }
    }
}
```

Problem 3: Recursion and the runtime stack

3-1)

mystery(20, 6)

```
-----
a = 20
b = 6
myst_rest = mystery(14, 6) = ...
-----
a = 14
b = 6
myst_rest = mystery(8, 6) = ...
-----
a = 8
b = 6
myst_rest = mystery(2, 6) = ...
    mystery(2,6)
    -----
    a = 2
    b = 6
    myst_rest = mystery(-4, 6) = ...
    -----
    return 1
    -----
    myst_rest = 1
    return 3
    -----
    myst_rest = 3
    return 5
    -----
    myst_rest = 5
    return 7
    -----
myst_rest = 7
return 9
```

3-2) mystery(20, 6) returns 9

3-3) including main(), there are 6 stack frames when we hit the base case (the mystery(-4, 6) call)

3-4) Any value of $a > 0$ and $b \leq 0$ would produce infinite recursion, then a would never decrease and could not meet the base case.

Problem 4: Rewriting a method

4-1)

```
public static boolean search(Object item, Object[] arr) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i].equals(item)) {  
            return true;  
        }  
    }  
    return false;  
}
```

4-2)

```
public static boolean Search(Object item, Object[] arr, int start)  
{  
    if (start >= arr.length) {  
        return false;  
    }  
  
    if (arr[start].equals(item)) {  
        return true;  
    }  
  
    return Search(item, arr, start + 1);  
}
```